GRUPO 3 · DESARROLLO WEB INTEGRADO

GYM TOBYFIT

Manual completo de ejecución y estudio

Visual Studio Code · MySQL · Maven · API REST · JWT · Pruebas

Una guía paso a paso para ejecutar, comprender y presentar el proyecto.

Versión con actualización estética de las cuatro páginas web.
Código de negocio y trabajos de los integrantes conservados.

Universidad Tecnológica del Perú
Ingeniería de Sistemas e Informática / Software
Sección 42003
Lima, Perú · 2026

GUÍA DE ESTUDIO / 01

Cómo usar este manual

El sistema combina una interfaz web y una API REST. La interfaz permite consultar productos, membresías y rutinas; la API también incluye administración de usuarios. Para una presentación completa, inicia Spring Boot y utiliza el navegador y Postman.

Tema	Página
Estructura y arquitectura	3
Preparar Visual Studio Code	4
Crear e importar MySQL	5
Variables y ejecución principal	6
Ensayo con perfil demo	7
Uso de las cuatro páginas	8
JWT y permisos	9
Objetos del proyecto	10
Endpoints REST	11–12
Postman y cuatro casos de seguridad	13–14
Maven y pruebas	15–16
Errores y diferencias previas	17
GitHub y presentación por integrante	18
Preguntas de estudio	19
Cambios y verificaciones	20

Ruta rápida para empezar

1. Instala JDK 21 y abre la carpeta gym-tobyfit-vs/gym-tobyfit en Visual Studio Code.

2. Si vas a utilizar MySQL, prepara una base de desarrollo e importa el SQL siguiendo la página 5.

3. Genera JWT_SECRET en la terminal PowerShell como se explica en la página 6. No omitas esta variable.

4. Ejecuta .\mvnw.cmd spring-boot:run y abre http://localhost:8080/index.html.

Para ensayar sin MySQL, usa el perfil demo de la página 7. Ese ensayo no sustituye la comprobación con MySQL que debe realizar el integrante 5.

Qué contiene el proyecto

Ubicación	Para qué sirve
gym-tobyfit-vs/gym-tobyfit/pom.xml	Declara Java 21, Spring Boot 3.4.10 y dependencias.
src/main/java/com/gym/gym_tobifit	Aplicación, controladores, entidades, servicios, repositorios y seguridad.
src/main/resources/static	Inicio, productos, membresías y rutinas; CSS, JavaScript visual y SVG.
src/main/resources/application*.properties	Configuración MySQL y perfil demo con H2.
src/test	Pruebas REST, JWT y perfil test.
gym_tobyfit.sql	Respaldo con productos y usuarios.
sql/01_usuarios_demo_jwt.sql	Cuentas ficticias ADMIN y USUARIO para MySQL.
gym_proyect.postman_collection.json	24 solicitudes originales de productos y usuarios.
scripts/probar_jwt.py	Comprobaciones HTTP contra un servidor en ejecución.
target/surefire-reports	Resultados incluidos en el ZIP original.
Documentación DOCX y diapositivas PDF	Material académico anterior; contiene descripciones que ya no coinciden con todo el código actual.
.github/workflows/maven.yml	Flujo de GitHub Actions para Java 21 y Maven.

Arquitectura en palabras simples

El navegador envía una petición HTTP. Si la ruta exige autenticación, Spring Security comprueba el JWT. El Controller recibe la petición; el Service realiza la operación; el Repository consulta o guarda mediante JPA; MySQL almacena los datos. La respuesta regresa como JSON.

Ejemplo: al consultar productos, productos.html envía GET /api/productos con Authorization: Bearer <token>. ProductoApiController llama a ProductoService, que consulta ProductoRepository y devuelve los productos.

La clase principal es GymTobifitApplication. Aunque la marca es TobyFit, el paquete y el artefacto usan gym_tobifit y gym-tobifit; conserva esos nombres al buscar archivos o ejecutar el JAR.

Preparar Visual Studio Code

Programas y requisitos

Elemento	Uso
JDK 21	Compilar y ejecutar el código Java. Java 17 no cumple la versión declarada.
Visual Studio Code	Abrir el código y ejecutar comandos en la terminal.
Extension Pack for Java	Ayuda con Java, Maven y ejecución de pruebas desde el editor.
Spring Boot Extension Pack	Opcional: facilita identificar y arrancar la aplicación.
MySQL o MariaDB local	Base de datos de la ejecución principal. El SQL procede de MariaDB 10.4.32.
Postman	Enviar peticiones y mostrar resultados HTTP.
Git	Revisar el estado local y preparar la entrega a GitHub.
Python 3	Opcional: ejecutar scripts/probar_jwt.py.

Abrir la carpeta correcta

1. Descomprime el ZIP completo en una carpeta de trabajo. No abras los archivos directamente dentro del ZIP.

2. En VS Code, selecciona Archivo > Abrir carpeta y abre gym-tobyfit-vs/gym-tobyfit. Ahí deben verse pom.xml, mvnw.cmd y src.

3. Si VS Code solicita confianza, concédela solamente después de reconocer que se trata de tu proyecto. Espera a que Java importe Maven.

4. Selecciona Terminal > Nueva terminal. Los comandos de este manual para Windows utilizan PowerShell.

```
java -version
.\mvnw.cmd -v
```

Comprueba que ambos comandos indiquen Java 21. Si usan otra versión, configura el JDK en el sistema y en “Java: Configure Java Runtime”; luego abre una nueva terminal.

No necesitas instalar Maven por separado

El proyecto incluye Maven Wrapper 3.9.11. mvnw.cmd lo utiliza en Windows; ./mvnw o bash mvnw lo utiliza en Linux/macOS. La primera ejecución necesita acceso a Maven Central para resolver dependencias.

Live Server puede mostrar HTML, pero no inicia Java, MySQL ni los endpoints. Para la demostración funcional abre la web desde el puerto 8080 de Spring Boot.

Preparar la base MySQL

La configuración predeterminada apunta a jdbc:mysql://localhost:3306/gym_tobyfit. Inicia tu servicio MySQL o el módulo MySQL de XAMPP, según tu instalación.

Importación con una interfaz gráfica

1. Abre phpMyAdmin o MySQL Workbench y conecta a tu servidor local.

2. Utiliza una base de desarrollo. El archivo gym_tobyfit.sql incluye DROP TABLE para productos y usuarios: importarlo reemplaza esas tablas y sus datos.

3. Importa gym_tobyfit.sql desde la raíz del proyecto. El archivo crea y selecciona gym_tobyfit, define productos y usuarios e inserta datos.

4. Después importa gym-tobyfit-vs/gym-tobyfit/sql/01_usuarios_demo_jwt.sql. Agrega las cuentas de demostración si no existen.

Alternativa desde el cliente mysql

Desde la raíz descomprimida del proyecto, abre el cliente. Ejecuta SOURCE dentro del cliente mysql; evita redirigir archivos con “<” en PowerShell.

```
mysql -u root -p
SOURCE gym_tobyfit.sql;
SOURCE gym-tobyfit-vs/gym-tobyfit/sql/01_usuarios_demo_jwt.sql;
USE gym_tobyfit;
SHOW TABLES;
SELECT COUNT(*) FROM productos;
SELECT id, correo, rol FROM usuarios;
```

Las otras dos tablas

El respaldo original contiene productos y usuarios. Membresia y Rutina sí existen en Java, pero sus tablas no están en ese SQL. Con spring.jpa.hibernate.ddl-auto=update, Hibernate las crea al iniciar con un usuario de base de datos que tenga permisos suficientes. Verifica nuevamente SHOW TABLES después del arranque.

Que el catálogo de membresías o rutinas esté vacío no significa que la autenticación falló. Crea datos de ensayo por Postman siguiendo la página 14.

Cuentas ficticias del script

Rol	Correo	Contraseña de ensayo
ADMIN	admin.jwt@tobyfit.local	DemoAdmin2026!
USUARIO	usuario.jwt@tobyfit.local	DemoUsuario2026!

Estas cuentas solo son para demostración. No supongas que las contraseñas de los usuarios antiguos del SQL sean las mismas.

Configurar y ejecutar con MySQL

Ejecuta estos pasos en la terminal PowerShell situada en la carpeta que contiene pom.xml. Las variables existen en esa terminal; si la cierras, tendrás que definirlas otra vez.

1. Generar una clave JWT compatible

```
$bytes = New-Object byte[] 32
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$rng.GetBytes($bytes)
$env:JWT_SECRET = [Convert]::ToBase64String($bytes)
$rng.Dispose()
```

Esto produce una clave aleatoria de 44 caracteres ASCII. Es compatible con los dos generadores existentes: JwtUtil exige HS256 y JwtService selecciona el algoritmo según la longitud de la clave. Mantén esta misma variable durante toda la sesión; cambiarla invalida los tokens anteriores.

2. Definir conexión y duración

```
$env:DB_URL = "jdbc:mysql://localhost:3306/gym_tobyfit"
$env:DB_USER = "root"
$env:DB_PASSWORD = "TU_CLAVE_MYSQL"
$env:JWT_EXPIRATION_MS = "3600000"
```

Sustituye TU_CLAVE_MYSQL por tu contraseña real. Si root no tiene contraseña en tu entorno local, usa una cadena vacía. No copies estos valores a GitHub.

3. Iniciar Spring Boot

```
.\mvnw.cmd spring-boot:run
```

Espera el mensaje de arranque de GymTobifitApplication y Tomcat en el puerto 8080. Deja esta terminal abierta. El sitio se abre en http://localhost:8080/index.html.

4. Comprobar y detener

Abre productos.html e inicia sesión con una cuenta demo. Después prueba http://localhost:8080/api/productos sin JWT: debe rechazar el acceso. Para comprobar con JWT usa Postman; abrir una URL en el navegador no agrega automáticamente el token al encabezado.

Para detener el servidor, presiona Ctrl+C en su terminal. Para ejecutar pruebas mientras está activo, abre otra terminal: Maven no necesita ese servidor para las pruebas MockMvc.

Linux/macOS

```
export JWT_SECRET="$(openssl rand -base64 32)"
export DB_URL="jdbc:mysql://localhost:3306/gym_tobyfit"
export DB_USER="root"
export DB_PASSWORD="TU_CLAVE_MYSQL"
bash mvnw spring-boot:run
```

Ensayar sin MySQL: perfil demo

El perfil demo usa H2 en memoria y ya está incluido en el proyecto. Permite ensayar la navegación, el login y productos sin instalar MySQL. Los datos desaparecen cuando se detiene la aplicación.

Arranque en Windows

1. Abre gym-tobyfit-vs/gym-tobyfit en VS Code y su terminal PowerShell.

2. Genera JWT_SECRET con el bloque de comandos de la página 6. Este paso también es necesario en demo para que el login de las páginas use un algoritmo aceptado por el filtro.

3. Ejecuta el perfil demo con Maven Wrapper.

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=demo"
```

4. Abre http://localhost:8080/index.html. Prueba las cuentas de ensayo de la página 5.

Qué datos se cargan

Dato	Comportamiento del perfil
Administrador	admin.jwt@tobyfit.local / DemoAdmin2026!
Usuario	usuario.jwt@tobyfit.local / DemoUsuario2026!
Producto	Polo deportivo demo, precio 59.90, activo.
Membresías y rutinas	No se cargan automáticamente. El catálogo puede estar vacío.
Persistencia	La base H2 se crea y elimina con el ciclo de la aplicación.

Crear datos de práctica

Usa Postman y un JWT para crear una membresía y una rutina con los ejemplos de la página 14. Esos datos solo pertenecen al ensayo; no son parte de la mejora estética ni fueron agregados al backend.

Volver a la ejecución principal

Detén el servidor con Ctrl+C. Inicia MySQL, configura DB_URL, DB_USER y DB_PASSWORD, y arranca con .\mvnw.cmd spring-boot:run, sin el argumento de perfil demo.

Diferencia entre demo y test

demo está pensado para utilizar el servidor manualmente. test es el perfil de las pruebas de integración y también utiliza H2. Ninguno acredita por sí solo que MySQL esté funcionando en tu computadora.

La clave predeterminada de demo es larga. Por la combinación actual de JwtService y JwtUtil, el token del login web puede ser rechazado. La variable JWT_SECRET aleatoria de 44 caracteres evita esa incompatibilidad sin editar el trabajo de seguridad.

Cómo usar las páginas del sistema

Página	Qué muestra y qué puedes hacer
/index.html	Portada, carrusel manual de servicios, tarjetas, catálogo integrado, acceso a membresías/rutinas y contacto.
/productos.html	Inicio de sesión, búsqueda por nombre/categoría/género, tarjetas con precio, actualización y cierre de sesión.
/membresias.html	Inicio de sesión y tarjetas de los planes activos con precio, duración y descripción.
/rutinas.html	Inicio de sesión y tarjetas con nombre, objetivo, nivel y duración en semanas.

Recorrido recomendado

1. Empieza en Inicio. Usa las flechas o los indicadores del carrusel para cambiar de servicio; el contenido no rota automáticamente.

2. Abre Productos. Sin sesión, se muestra el panel de acceso. Ingresa las credenciales de una cuenta válida.

3. Comprueba que las tarjetas provienen de la API. Busca una palabra del catálogo; usa Actualizar para consultar de nuevo.

4. Abre Membresías. Solo se muestran planes con activa === true. Si aparece una lista vacía, revisa los registros.

5. Abre Rutinas. Esta página conserva su propio almacenamiento de sesión; puede pedirte el login otra vez.

6. Prueba Cerrar sesión y vuelve a entrar. En celular utiliza el botón de menú para navegar.

Comportamientos que se conservaron

Productos y membresías guardan el JWT con la clave “token”. Rutinas usa “tobyfit_token”. Cerrar sesión en una de esas claves no elimina la otra. Esta diferencia pertenece al código original y se conserva; no se agregó una sesión global nueva.

Las páginas consultan información y no incluyen formularios de administración CRUD. Para crear, actualizar o eliminar registros, usa los endpoints desde Postman con los permisos correspondientes.

El enlace de contacto mantiene contacto@tobyfit.local, una dirección de demostración. Las fotografías de productos siguen dependiendo de Unsplash y no necesariamente corresponden a imagenPrincipal; si fallan, el código presenta un respaldo visual.

Mejora estética aplicada

Se unificaron encabezado, pie, tipografía, colores, botones, formularios y tarjetas. Se agregó un carrusel y una ilustración SVG local en la portada. El CSS incluye reglas para pantallas pequeñas, foco de teclado y preferencia de movimiento reducido.

Entender JWT y los permisos

El flujo de autenticación

1. El cliente envía correo y contraseña a un endpoint público de login.

2. AuthenticationManager utiliza MyUserDetailsService para buscar al usuario en la base y comprobar la contraseña con BCrypt.

3. Si las credenciales son válidas, un generador firma un JWT con el correo como subject y una fecha de expiración.

4. El cliente envía el JWT en cada solicitud protegida mediante Authorization: Bearer <token>.

5. JwtFilter valida el token con JwtUtil, consulta nuevamente al usuario y asigna sus permisos. El endpoint ejecuta la operación autorizada.

Los dos logins del proyecto

Ruta	Entrada	Respuesta
POST /auth	username (correo) y password	200; token como texto. Usa JwtUtil y HS256 explícito.
POST /api/auth/login	correo y contrasena	200; JSON con token. Usa JwtService; lo utilizan las páginas web.

No envíes los campos de una ruta a la otra. AuthRequest tiene validaciones de campos; LoginRequest no contiene esas mismas anotaciones.

Qué significa cada estado

Estado	Interpretación
200	Solicitud realizada correctamente.
201	Membresía o rutina creada.
204	Membresía o rutina eliminada sin cuerpo de respuesta.
400	Solicitud o JSON inválido en los casos controlados.
401	Falta autenticación o el token/credenciales no son válidos.
403	La autenticación es válida, pero falta el rol requerido.
404	Registro inexistente en los casos contemplados por el controlador.

Permisos reales de esta versión

Productos: lectura con autenticación; creación, actualización y eliminación requieren ADMIN. Usuarios: todas sus operaciones requieren ADMIN. Membresías y rutinas: exigen autenticación por la regla global, pero sus controladores no agregan un requisito ADMIN. No afirmes que todas las escrituras están restringidas al administrador.

La aplicación es stateless: no conserva una sesión de servidor entre peticiones. JwtUtil usa JWT_EXPIRATION_MS; JwtService conserva una duración fija de una hora. El JWT no incluye la contraseña y Usuario no la devuelve en el JSON.

Los cuatro objetos REST

El código actual expone Producto, Usuario, Membresia y Rutina. Cada objeto tiene más de cuatro atributos. No se agregó ni se cambió ninguno para el rediseño.

Objeto	Atributos Java y tipos
Producto · 9	id: Long; nombre: String; categoria: String; genero: String; precio: BigDecimal; imagenPrincipal: String; imagenes: String; modelos: String; activo: Boolean.
Usuario · 8	id: Long; nombre: String; apellido: String; telefono: String; edad: Integer; correo: String; contrasena: String; rol: Rol (USUARIO o ADMIN).
Membresia · 6	id: Long; nombre: String; duracionDias: Integer; precio: Double; descripcion: String; activa: Boolean.
Rutina · 6	id: Long; nombre: String; objetivo: String; nivel: String; duracionSemanas: Integer; descripcion: String.

Detalles para explicar en clase

Las clases con @Entity se relacionan con tablas; @Id identifica la clave primaria y @GeneratedValue indica que la base genera el identificador. Los repositorios JpaRepository proporcionan operaciones de consulta y persistencia sin escribir todo el SQL manualmente.

Producto utiliza BigDecimal para precio; Membresia utiliza Double. Son tipos diferentes que se conservaron. imagenes y modelos se almacenan como cadenas en columnas TEXT; no los describas como listas Java si vas a explicar esta implementación.

Usuario tiene ocho atributos en la entidad. contrasena es WRITE_ONLY al serializar: puede entrar en JSON, pero no aparece en la respuesta. Además, el servicio de actualización de usuarios no cambia la contraseña desde PUT; hay un método interno de restablecimiento sin una ruta REST expuesta en este controlador.

Objeto del dominio y DTO no son lo mismo

Las cuatro entidades representan información del gimnasio. AuthRequest, LoginRequest y LoginResponse son DTO para intercambiar datos de autenticación; tienen dos, dos y un atributo respectivamente. Si el profesor exige específicamente cuatro DTO, esa es una condición diferente de cuatro objetos del dominio con cuatro atributos.

Material académico anterior

El DOCX original menciona CarritoCompras y dos controladores REST como parte de una versión anterior. En el código activo de este ZIP, el cuarto objeto es Rutina y existen cuatro controladores del dominio. El material anterior se conserva; explica la versión ejecutable con las clases actuales.

Endpoints: productos y usuarios

Base de la API: http://localhost:8080. En solicitudes con cuerpo JSON utiliza Content-Type: application/json; en rutas protegidas agrega Authorization: Bearer <token>.

Método y ruta	Rol	Resultado implementado
GET /api/productos	Autenticado	200 con lista, incluso si está vacía.
GET /api/productos/{id}	Autenticado	200 con producto; 404 si no existe.
POST /api/productos	ADMIN	200 con producto guardado. No devuelve 201 en esta versión.
PUT /api/productos/{id}	ADMIN	200 al actualizar. El servicio exige un producto existente; no hay respuesta 404 explícita en el controlador para la excepción.
DELETE /api/productos/{id}	ADMIN	200 al eliminar. No devuelve 204 en esta versión.
GET /api/usuarios	ADMIN	200 con lista; contrasena no se serializa.
PUT /api/usuarios/{id}	ADMIN	200; 400 si el ID del cuerpo contradice la ruta; 404 cuando el servicio lanza una RuntimeException.
DELETE /api/usuarios/{id}	ADMIN	200; 404 cuando el servicio lanza una RuntimeException.

Ejemplo de producto completo

```
{
  "nombre": "Mancuerna de ensayo",
  "categoria": "Pesas",
  "genero": "Unisex",
  "precio": 89.90,
  "imagenPrincipal": "mancuerna.jpg",
  "imagenes": "[]",
  "modelos": "[]",
  "activo": true
}
```

Ejemplo de actualización parcial de usuario

```
{ "nombre": "Nombre de ensayo", "telefono": "999888777" }
```

En Usuario, el controlador usa el ID de la ruta si el cuerpo no lo trae. En Producto, PUT asigna el ID de la ruta y el servicio copia únicamente campos no nulos. En membresías y rutinas el comportamiento de actualización es distinto; revisa la página siguiente.

No hay POST /api/usuarios ni GET /api/usuarios/{id} en el controlador actual. No presentes esos endpoints como funciones implementadas. Las ocho operaciones de esta página son las que cubren los 24 @Test originales.

Endpoints: membresías y rutinas

Método y ruta	Respuesta del controlador
GET /api/membresias	200; lista de membresías.
GET /api/membresias/{id}	200 si existe; 404 si no existe.
POST /api/membresias	201; ID generado y membresía guardada.
PUT /api/membresias/{id}	200 si existe; 404 si no existe.
DELETE /api/membresias/{id}	204 si existe; 404 si no existe.
GET /api/rutinas	200; lista de rutinas.
GET /api/rutinas/{id}	200 si existe; 404 si no existe.
POST /api/rutinas	201; ID generado y rutina guardada.
PUT /api/rutinas/{id}	200 si existe; 404 si no existe.
DELETE /api/rutinas/{id}	204 si existe; 404 si no existe.

Estas diez operaciones requieren un JWT válido. El código actual no agrega @PreAuthorize para exigir ADMIN; un usuario autenticado puede llegar a esos controladores.

Actualizar exige completar los campos

Los PUT de membresías y rutinas copian todos los campos del cuerpo. Usa el objeto completo al actualizar: si omites campos, podrían quedar nulos y provocar errores por las restricciones de la base. No asumas el mismo comportamiento parcial de Producto o Usuario.

Distinguir API y visualización

GET /api/membresias devuelve la lista del servicio. La página membresias.html aplica el filtro activa === true para mostrarla. Un plan inactivo puede existir en la API y estar oculto en la web. Productos aplica un filtro visual activo !== false; Rutinas muestra la lista recibida.

Recuento correcto

Hay 18 operaciones del dominio: 5 de productos, 3 de usuarios, 5 de membresías y 5 de rutinas. A ellas se suman 2 rutas de autenticación. Se cuentan combinaciones de método HTTP y patrón de ruta; múltiples IDs no son operaciones diferentes.

Los controladores llaman a sus servicios y repositorios. La interfaz consume el login y los listados; el resto se demuestra con Postman o las pruebas automatizadas.

Postman: demostrar seguridad

Preparar la herramienta

1. Inicia el servidor y comprueba que sea el puerto 8080. Importa gym_proyect.postman_collection.json desde Postman > Import.

2. La colección original incluye 24 solicitudes de productos/usuarios, pero no incluye una autenticación global ni las rutas de membresías/rutinas. Crea solicitudes adicionales para ensayar; conserva la colección original.

3. Para cada petición de login selecciona Body > raw > JSON y Authorization > No Auth.

Caso 1: login correcto

```
POST http://localhost:8080/auth
Content-Type: application/json

{
  "username": "admin.jwt@tobyfit.local",
  "password": "DemoAdmin2026!"
}
```

Esperado: 200 y un JWT como texto. Copia el texto completo, sin comillas ni espacios añadidos. Si pruebas /api/auth/login, usa correo y contrasena; la respuesta esperada es un objeto JSON con token.

Caso 2: login incorrecto

Repite POST /auth con password = incorrecta. Esperado: 401; no debe entregarse un token. Una solicitud incompleta o JSON mal formado a /auth devuelve 400, un caso distinto.

Caso 3: endpoint sin JWT

Envía GET http://localhost:8080/api/productos con No Auth y sin encabezado Authorization heredado. Esperado: 401. Este caso demuestra que visitar la URL no basta para acceder.

Caso 4: endpoint con JWT

En la misma solicitud selecciona Authorization > Bearer Token y pega solo el JWT del caso 1. Postman construye Authorization: Bearer <token>. Envía la petición: esperado 200 y una lista de productos.

Permisos por rol

Obtén un JWT de usuario.jwt@tobyfit.local con DemoUsuario2026!. GET /api/productos debe responder 200; GET /api/usuarios debe responder 403. Con JWT de ADMIN, GET /api/usuarios debe responder 200 y no mostrar contrasena.

Captura la ruta, método, código y respuesta de cada caso. No confundas 401 con 403. Evita publicar tokens vigentes en capturas abiertas o en el repositorio.

Postman: crear datos y hacer CRUD

Crear una membresía de ensayo

```
POST http://localhost:8080/api/membresias
Authorization: Bearer <token>
Content-Type: application/json

{
  "nombre": "Plan mensual de ensayo",
  "duracionDias": 30,
  "precio": 99.90,
  "descripcion": "Acceso mensual al gimnasio",
  "activa": true
}
```

Esperado: 201 y el objeto con id. Guarda ese ID; usa GET /api/membresias/{id} para consultarlo. Refresca la página de membresías para ver la tarjeta.

Crear una rutina de ensayo

```
POST http://localhost:8080/api/rutinas
Authorization: Bearer <token>
Content-Type: application/json

{
  "nombre": "Rutina inicial de ensayo",
  "objetivo": "Fuerza",
  "nivel": "Principiante",
  "duracionSemanas": 4,
  "descripcion": "Plan de entrenamiento inicial"
}
```

Esperado: 201 y el objeto con id. Abre Rutinas, inicia sesión allí si lo solicita y pulsa Actualizar rutinas.

Recorrido completo con un registro de ensayo

1. Crear: envía POST y conserva el ID devuelto; no supongas que siempre será 1.

2. Leer: envía GET con ese ID. Comprueba que los valores coincidan con el cuerpo enviado.

3. Actualizar: envía PUT al mismo ID con todos los campos; cambia la descripción. Esperado: 200.

4. Eliminar: envía DELETE al mismo ID de ensayo. En membresías y rutinas se espera 204. Luego GET a ese ID debe responder 404.

Haz las operaciones de eliminación sobre datos de ensayo identificados. La colección original contiene eliminaciones con IDs fijos, como /api/usuarios/1; no ejecutes toda la colección sin revisar sus solicitudes.

Maven y los 24 @Test originales

Ejecuta las pruebas desde la carpeta del pom.xml. Maven compila Java y ejecuta JUnit; no necesitas abrir Postman ni mantener el servidor manual en el puerto 8080 para MockMvc.

```
.\mvnw.cmd clean test
.\mvnw.cmd "-Dtest=ApiControllersTest" test
.\mvnw.cmd clean verify
```

clean elimina artefactos compilados anteriores; test ejecuta pruebas; verify realiza la fase de verificación. Para empaquetar y probar usa .\mvnw.cmd clean package. Evita omitir pruebas al preparar la exposición.

Operación	Casos existentes en ApiControllersTest	N.°
Listar productos	test01 lista; test02 lista vacía; test03 varios productos	3
Buscar producto	test04 ID existente; test05 inexistente; test06 otro ID	3
Crear producto	test07 creación; test08 categoría; test09 estado activo	3
Actualizar producto	test10 actualización; test11 precio; test12 estado	3
Eliminar producto	test13 ID; test14 otro ID; test15 tercer ID	3
Listar usuarios	test16 lista; test17 vacía; test18 varios usuarios	3
Actualizar usuario	test19 actualización; test20 nombre; test21 apellido	3
Eliminar usuario	test22 ID; test23 otro ID; test24 tercer ID	3

Qué acredita esta suite

Hay 24 @Test, distribuidos en 8 operaciones, con 3 casos por operación: cumple el mínimo de dos casos para esas ocho. Los casos de eliminación usan IDs distintos y verifican HTTP 200; no son necesariamente un caso positivo y uno negativo ni una comprobación de borrado real en MySQL.

ApiControllersTest usa @WebMvcTest, servicios simulados con Mockito y addFilters=false. Comprueba respuestas de controladores; no evalúa la cadena JWT. Las pruebas de seguridad están en una clase distinta.

Interpretar el reporte

Un resultado correcto muestra BUILD SUCCESS y cero failures/errors. Los archivos TXT y XML se generan en target/surefire-reports. Para una evidencia válida guarda el resultado de tu ejecución actual, con su fecha; los reportes del ZIP son evidencia previa.

Pruebas adicionales y seguridad

Clase	Ejecuciones en los reportes originales	Alcance
ApiControllersTest	24	Productos y usuarios: 8 operaciones, 3 casos por operación.
MembresiaRutinaApiTest	20	Membresías y rutinas: 10 operaciones, 2 casos por operación.
JwtSecurityIntegrationTest	26	Cadena de seguridad y repositorios H2 reales; casos parametrizados incluidos.
JwtUtilTest	8	Generación/validación, firmas, campos, algoritmo y configuración del token.
Total incluido en el ZIP	78	Reportes con cero fallos, errores y omitidos. No equivale a una nueva ejecución.

Dos casos por cada operación adicional

Para cada uno de Membresia y Rutina: listar con datos/lista vacía; buscar existente/inexistente; crear con objeto/JSON inválido; actualizar existente/inexistente; eliminar existente/inexistente. Son 5 operaciones × 2 casos × 2 objetos = 20 pruebas REST.

Cuatro casos pedidos al integrante 4

Caso	Prueba existente que lo demuestra
Login correcto	loginGeneraTokenFirmadoDesdeBaseDeDatos
Login incorrecto	claveIncorrectaNoEntregaToken; usuarioDesconocidoNoEntregaToken
Endpoint sin JWT	productosSinTokenRetorna401; usuariosSinTokenRetorna401
Endpoint con JWT	productosConTokenDeLoginRetorna200

La suite de integración añade permisos ADMIN/USUARIO, ausencia de contraseñas en respuestas, tokens alterados y vencidos, usuario eliminado, cambio de rol y ausencia de sesión de servidor. Las dos pruebas parametrizadas producen 5 y 4 ejecuciones; por eso 17 @Test + 2 métodos parametrizados equivalen a 26 ejecuciones.

Ejecutar por clase

```
.\mvnw.cmd "-Dtest=MembresiaRutinaApiTest" test
.\mvnw.cmd "-Dtest=JwtSecurityIntegrationTest" test
.\mvnw.cmd "-Dtest=JwtUtilTest" test
```

Comprobar el servidor real por HTTP

```
python scripts/probar_jwt.py --output evidencias_http.json
```

Inicia antes Spring Boot con las cuentas demo. Este script usa /auth, comprueba tokens y roles y escribe un JSON de resultados. No verifica por sí solo el contrato /api/auth/login de las páginas; prueba también ese login con Postman.

Resolver problemas antes de presentar

Síntoma	Qué revisar
No se encuentra pom.xml	Terminal en gym-tobyfit-vs/gym-tobyfit; no en static ni solo en la raíz del ZIP.
Release version 21 not supported	JDK activo. java -version y mvnw.cmd -v deben usar Java 21.
Could not resolve JWT_SECRET	Genera la variable en la misma terminal antes de iniciar; el archivo de error original muestra precisamente este problema.
Login entrega token y luego aparece 401	Verifica el algoritmo: JwtService selecciona HS384/HS512 con claves largas, pero JwtUtil exige HS256. Usa la clave aleatoria de 44 caracteres y vuelve a iniciar sesión.
Access denied / Communications link failure	MySQL encendido, puerto correcto y DB_USER/DB_PASSWORD válidos.
Unknown database gym_tobyfit	Importa el SQL en la base de desarrollo correcta.
Port 8080 already in use	Detén el arranque anterior o identifica qué proceso utiliza ese puerto.
Maven no descarga dependencias	Revisa internet/acceso a Maven Central. Es un problema de resolución, no un resultado de las pruebas.
Membresías o rutinas vacías	Crea datos por Postman y verifica activa=true en membresías.
Rutinas vuelve a pedir login	Su clave localStorage es distinta; comportamiento original conservado.
Productos sin fotografía	El código usa Unsplash; requiere conexión y presenta un respaldo si la imagen falla.
401 al ejecutar Postman	Revisa Bearer, token completo, vigencia y la clave con la que arrancó el servidor.
403 en /api/usuarios	Usa una cuenta ADMIN; USUARIO no tiene ese permiso.

Diferencias de la versión que debes conocer

El respaldo SQL no contiene membresías/rutinas; Hibernate debe crearlas. La colección Postman no tiene login global ni pruebas de esos dos objetos. El DOCX y PDF anteriores describen una etapa distinta. Estos archivos y los comportamientos originales se conservaron.

Un JWT emitido por /api/auth/login con una clave larga puede no ser aceptado por el filtro, aunque /auth funcione. Las pruebas incluidas se concentran en /auth. Para la presentación verifica ambos contratos usando la misma clave compatible.

Conserva el mismo JWT_SECRET entre reinicios si quieres reutilizar un token. Si generas una clave nueva, elimina la sesión de la web o inicia sesión nuevamente.

GitHub y presentación por integrante

Verificar el repositorio desde la raíz

La raíz contiene .git y .github; es la carpeta superior a gym-tobyfit-vs. El repositorio configurado es EspinozaBerriosJulioCesar/GRUPO-3--APF2-GYM-TOBYFIT-VS. La configuración local no demuestra que los cambios actuales estén publicados.

```
git status --short
git remote -v
git branch --show-current
git fetch origin
git diff --name-status origin/main
```

En el ZIP original ya existían numerosos cambios locales pendientes. Revisa qué pertenece a cada integrante antes de preparar commits. El rediseño no reemplazó ni reinició el historial; tampoco se publicaron cambios en GitHub desde esta revisión.

Preparar la entrega cuando el equipo la valide

Agrega los archivos aprobados de forma explícita. La carpeta visual se puede preparar con el comando siguiente; revisa además si faltan cambios del backend de tus compañeros antes de publicar.

```
git add gym-tobyfit-vs/gym-tobyfit/src/main/resources/static
git diff --cached --stat
git commit -m "Mejora visual de TobyFit"
git push origin NOMBRE_DE_TU_RAMA
```

Sustituye NOMBRE_DE_TU_RAMA por la rama confirmada con el equipo. Después abre GitHub, comprueba el commit y revisa Actions. El workflow incluido ejecuta mvn -B clean verify en Java 21 para main y jwt-seguridad en los eventos declarados.

Integrante	Qué mostrar en la exposición
1 · JWT y seguridad	Login, generación del JWT, validación, solicitud sin token y con token; explicar BCrypt y Bearer.
2 · Objetos REST	Las cuatro entidades y sus atributos; rutas que reciben o devuelven cada objeto.
3 · Pruebas REST	Los 24 casos originales y los 20 adicionales; distribución de casos por operación.
4 · Pruebas de seguridad	Los cuatro casos requeridos y la suite JwtSecurityIntegrationTest.
5 · Revisión final	Arranque en VS Code, conexión MySQL, Maven, Postman, estado GitHub y revisión de errores.

Orden de demostración: inicio visual → login → productos → membresías/rutinas → Postman sin/con JWT → Maven → MySQL → GitHub. Usa datos de ensayo y lleva capturas de los resultados reales.

Preguntas para estudiar y sustentar

¿Qué problema resuelve el proyecto?

Centraliza la consulta y gestión del catálogo, usuarios, membresías y rutinas del gimnasio mediante una aplicación web y servicios REST.

¿Qué es una API REST?

Un conjunto de rutas HTTP que permite intercambiar datos. Aquí GET consulta, POST crea o autentica, PUT actualiza y DELETE elimina.

¿Por qué hay Controller, Service y Repository?

Separan la recepción HTTP, las operaciones del sistema y el acceso a datos. Así cada capa tiene una responsabilidad más clara.

¿Qué hace un JWT?

Acredita una autenticación mediante un token firmado. El servidor comprueba su firma, vencimiento y usuario antes de aceptar una petición.

¿JWT cifra la contraseña?

No. El token de esta aplicación no incluye la contraseña. BCrypt se usa para verificar el hash de la contraseña almacenada.

¿Por qué sin token devuelve 401 y con USUARIO puede dar 403?

401 indica que falta autenticación válida. 403 indica que ya existe autenticación, pero el usuario no posee el permiso requerido.

¿Qué se prueba con MockMvc?

Solicitudes y respuestas de la aplicación sin levantar un servidor HTTP externo. Algunas suites simulan servicios; la integración de seguridad utiliza H2 y filtros reales.

¿Por qué 78 ejecuciones y no solo 24?

Hay 24 REST originales, 20 REST adicionales, 26 ejecuciones de seguridad y 8 de JwtUtil. Los casos parametrizados se cuentan por cada entrada.

¿Qué cambió en esta entrega?

La presentación HTML/CSS, navegación, carrusel, encabezado, pie y una ilustración. Java, JWT, SQL, Maven, Postman y pruebas se mantuvieron.

¿Es lo mismo demo, test y MySQL?

No. demo sirve para ensayo manual con H2; test configura pruebas con H2; la ejecución principal usa MySQL y sus variables de conexión.

¿Las páginas administran todo el CRUD?

No. Las páginas consultan los catálogos; las operaciones de creación, modificación y eliminación se muestran mediante Postman/API.

¿Qué evidencia confirma que el proyecto está listo?

Un arranque real, datos en MySQL, resultados actuales de Maven y Postman y el commit aprobado en GitHub. Un archivo adjunto anterior no sustituye esas comprobaciones.

Cambios y verificación de esta entrega

Archivos de presentación modificados

Archivo dentro de static	Cambio
index.html	Portada, carrusel manual, tarjetas de servicios, catálogo integrado y bloques de contacto.
productos.html	Encabezado/pie comunes y aplicación del diseño al catálogo y login.
membresias.html	Encabezado/pie comunes y estilo de formularios y planes.
rutinas.html	Encabezado/pie comunes y estilo de formularios y rutinas.
css/tobyfit.css · nuevo	Sistema visual compartido y reglas de adaptación a pantallas.
js/visual.js · nuevo	Menú y carrusel; no realiza consultas API ni modifica localStorage.
images/training.svg · nuevo	Ilustración local de equipamiento para la portada.

Protección del trabajo de los integrantes

La comparación contra el ZIP original confirma igualdad byte a byte de Java, pruebas, propiedades, pom.xml, SQL, colección Postman, script JWT e historial Git. Los bloques JavaScript originales de login y consultas de productos/membresías/rutinas conservan exactamente el mismo texto.

Validaciones realizadas

Se revisaron código, configuración, SQL, colección Postman, documentos originales, reportes Surefire y estado Git local. Se comprobó la sintaxis de los scripts y se verificaron login correcto/incorrecto, tarjetas, búsqueda, lista vacía, token rechazado y logout en un DOM de prueba con respuestas simuladas; esto verifica la integración de la interfaz, no el servidor Java.

Los reportes suministrados registran 78 ejecuciones sin fallos. La nueva ejecución de Maven no pudo resolver el parent de Spring Boot por acceso a Maven Central. El entorno disponible tiene Java 17, no Java 21, y no dispone de MySQL configurado; por ello no se certifica una nueva ejecución completa del backend.

La comprobación visual en un navegador no se completó: no había Chromium local disponible y el navegador remoto no pudo acceder al servidor de trabajo. Se revisó estáticamente el CSS y la estructura de la interfaz; el equipo debe abrir las cuatro páginas en computadora y celular antes de presentar.

Lista final del integrante 5

Confirmar Java 21; iniciar MySQL e importar datos; definir JWT_SECRET compatible; iniciar Spring Boot; probar ambos logins; consultar los tres catálogos; ejecutar clean verify; demostrar los cuatro casos JWT en Postman; comprobar datos persistidos; revisar GitHub y Actions; capturar evidencias actuales.

La calidad visual ayuda a la presentación, pero la nota depende de la rúbrica y de las verificaciones reales que muestre el equipo.